Bytes Speak All Languages: Cross-Script Name Retrieval via Contrastive Learning

Vedant Vivek Jumle
v.v.jumle@student.tudelft.nl
Delft University of Technology
Netherlands

Carolyn Alcaraz
C.Alcaraz-1@student.tudelft.nl
Delft University of Technology
Netherlands

Gonenc Turanli
G.Turanli-1@student.tudelft.nl
Delft University of Technology
Netherlands

Ceylin Ece
C.Ece-1@student.tudelft.nl
Delft University of Technology
Netherlands

Abstract

Retrieving person names across different writing systems is a fundamental challenge in multilingual information retrieval: a query like "Schwarzenegger" should match its Arabic, Russian, or Chinese phonetic equivalents, despite sharing no characters. Existing approaches based on edit distance or phonetic hashing fail entirely once the script boundary is crossed.

We present a byte-level transformer encoder trained with InfoNCE contrastive loss and Approximate Nearest Neighbour hard-negative mining. The model operates directly on raw UTF-8 bytes—requiring no tokeniser, no language-specific preprocessing, and no script detection. We construct a dataset of 119,040 named entities with 4.67 million positive pairs spanning 8 non-Latin scripts (Arabic, Russian, Chinese, Japanese, Hebrew, Hindi, Greek, Korean), generated via LLM-based phonetic transliteration.

Our model achieves 0.775 MRR and 0.897 R@10 overall, reducing the R@10 script gap—the difference between Latin and non-Latin query performance—from 0.93 (edit-distance baselines) to 0.096, a 10× reduction, while outperforming the strongest baseline (Transliterate, 0.565 MRR) by 37%.

CCS Concepts

• **Information systems** → **Retrieval models and ranking**; *Similarity measures*; • **Computing methodologies** → *Neural networks*.

Keywords

cross-script retrieval, phonetic embeddings, byte-level encoding, contrastive learning, name matching

ACM Reference Format:
Vedant Vivek Jumle, Carolyn Alcaraz, Gonenc Turanli, and Ceylin Ece. 2026. Bytes Speak All Languages: Cross-Script Name Retrieval via Contrastive Learning. In *Proceedings of Information Retrieval (DSAIT4050 '25).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Retrieving a person by name is deceptively hard when the query and the indexed record differ in script. A sanctions screening system must match *Владимир Путин* against *Vladimir Putin*; a border control system must link *Αλέξανδρος* to *Alexandros* or *Aleksander*. In each case the same individual is referred to, yet the surface forms share no characters. Classical approaches—Levenshtein edit distance [7, 10], double metaphone, and BM25 [11]—operate on raw character sequences and fail completely once a script boundary is crossed.

Recent work has shown that even strong multilingual dense retrievers degrade significantly when queries are transliterated rather than written in their native script [1]. This *script gap* is most severe for short name entities, where there is no surrounding semantic context to compensate for surface-level mismatch. Rule-based transliteration pipelines (e.g., ICU transliterate) partially close the gap, but apply a fixed canonical mapping that cannot capture the phonetic variation introduced by different romanization conventions or LLM-generated phonetic perturbations.

We address the specific problem of **cross-script phonetic name retrieval**: given a query name in any script or phonetic form, retrieve the correct named entity from a corpus indexed by its English name. We propose a lightweight transformer encoder trained from scratch on raw UTF-8 bytes [14, 15]. Each character in any writing system decomposes deterministically into 1–4 bytes from a fixed 256-token vocabulary, eliminating out-of-vocabulary tokens entirely and requiring no script detection or language-specific preprocessing. The encoder is trained with InfoNCE contrastive loss [13] and ANCE-style hard-negative mining [12], pushing phonetically equivalent names together and phonetically distinct names apart in embedding space.

To train and evaluate the model, we construct a dataset of 119,040 named entities with 4.67 million positive name pairs covering 8 non-Latin scripts (Arabic, Russian, Chinese, Japanese, Hebrew, Hindi, Greek, Korean), generated via LLM-based phonetic transliteration and augmented with Wikidata ground truth. Evaluation distinguishes three query types: *phonetic* (Latin variant of the English name), *script* (direct transliteration into a non-Latin script), and *combined* (phonetic Latin variant transliterated into a non-Latin script).

Our main contributions are: (1) a controlled benchmark for cross-script and phonetically perturbed name retrieval; (2) a compact byte-level contrastive encoder that achieves 0.897 R@10 and 0.775 MRR overall, reducing the script gap in R@10 from 0.93 to 0.096—a 10× reduction over edit-distance baselines; and (3) a systematic comparison of classical string-similarity baselines, rule-based transliteration, and dense retrieval, with a FAISS index ablation characterising the recall–latency tradeoff [5, 9].

DSAIT4050 '25, Delft, Netherlands
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

2 Related Work

Fuzzy name matching. Classical name matching relies on character-level edit distance [7, 10] and phonetic codes such as double metaphone and Metaphone, which encode names by their approximate English pronunciation. BM25 [11] with character *n*-grams provides a sparse retrieval baseline. These methods are effective within a single script but fail categorically across script boundaries: a Latin query and an Arabic document share no character-level overlap regardless of phonetic similarity.

Script gap in neural IR. Chari et al. [1] show that strong multilingual dense retrievers such as BGE-M3 [2] degrade severely when queries are transliterated rather than written in their native script. Their mitigation strategy fine-tunes existing models on mixtures of native and Latinised text, requiring large pretrained multilingual backbones. Liu et al. [8] (TransliCo) address the same barrier by contrasting sentence representations with their Latin transliterations during pretraining, improving cross-script alignment in general NLP tasks. Both works treat the script gap as a fine-tuning or pretraining problem for sentence-level encoders. We instead ask whether it can be solved *from scratch*, for the specific case of short name entities where no semantic context is available to compensate for surface mismatch.

Contrastive dense retrieval. Karpukhin et al. [6] (DPR) establish the dual-encoder paradigm, training encoders with in-batch negatives to maximise query–document similarity. Gao et al. [4] (SimCSE) demonstrate that hard negatives substantially improve contrastive embedding quality, and Robinson et al. [12] provide grounding for hard-negative selection. The InfoNCE objective [13] underlies our training loss. We adopt the contrastive retrieval framework but train entirely from scratch on raw UTF-8 bytes [15], without inheriting pretrained representations—motivated by the incompatibility of subword vocabularies with cross-script phonetic alignment.

3 Dataset and Splits

We construct a dataset of 119,040 person-name entities with approximately 4.67 million positive pairs spanning 8 non-Latin scripts (Arabic, Russian, Chinese, Japanese, Hebrew, Hindi, Greek, Korean), seeded from Wikidata and expanded via a four-stage LLM pipeline described in Section 4.

3.1 Entity Collection

Entities are seeded from Wikidata, which provides canonical English names along with partial cross-script labels. The 119,040 entities are drawn via stratified sampling by script-coverage bucket (0, 1–2, 3–4, 5+ non-English labels), preventing English-only names from dominating the training distribution.

3.2 Positive Pair Generation

Each positive pair is tagged as *phonetic* (Latin spelling variant of the English anchor), *script* (cross-script transliteration), or *combined* (phonetic Latin variant then transliterated into a non-Latin script). Only 0.5% of pairs come from Wikidata ground truth; the remaining 99.5% are LLM-generated, capturing realistic phonetic variation that rule-based systems cannot reproduce. Romanization ambiguity for Japanese and Korean introduced conflicting training signal, contributing to lower retrieval performance on those scripts.

3.3 Train-Eval-Test Split

Splits are assigned strictly at the entity level (80/10/10, deterministic MD5 hash of entity ID): all variants of an entity across all scripts go to a single partition, preventing leakage through shared phonetic structure and ensuring generalization to entirely unseen identities at test time.

4 Methodology

The encoder is trained in three stages: LLM-driven data augmentation, byte-level contrastive training, and retrieval evaluation.

4.1 LLM Augmentation Pipeline

The four-stage pipeline transforms the raw 2M-entity Wikidata name set into a training-ready dataset. Stage 1 applies stratified sampling by script-coverage bucket (0, 1–2, 3–4, 5+ non-English labels) to prevent English-only names from dominating. Stage 2 generates phonetic spelling variants of each English anchor using `Llama-3.1-8B-Instruct` (pronunciation-preserving variants only; no abbreviations or initials). Stage 3 transliterates each anchor and its Latin variants into the eight target scripts using `Qwen3-Coder-30B-A3B-Instruct-FP8` via the TU Delft TULIP API with structured JSON output. Stage 4 merges Wikidata ground-truth labels with LLM output, deduplicates, tags each positive as *phonetic*/*script*/*combined*, precomputes UTF-8 byte sequences, and assigns entity-level splits.

4.2 Byte-level Contrastive Training

4.2.1 Architecture. A lightweight transformer encoder (6 layers, 8 heads, hidden dim 256, FFN dim 1024, dropout 0.1, max length 256) is trained from scratch on raw UTF-8 bytes. Names are tokenized into bytes (vocabulary size 256), embedded, and summed with positional embeddings. The encoder output is mean-pooled over non-padding positions and L2-normalized, yielding one unit vector per name.

4.2.2 Training Objective and Hard Negatives. English anchor names serve as queries; a sampled positive variant is the target. Training uses InfoNCE loss with in-batch negatives. During a random-batch warmup phase the model develops initial similarity structure; afterward, ANCE-style hard negatives (nearest non-matching neighbors retrieved via a periodically updated FAISS index) are blended in alongside random negatives, concentrating gradient on the model's current failure cases.

4.3 Retrieval Evaluation

The corpus consists of canonical English anchor names encoded into a FAISS index as L2-normalized embeddings. Each test-set positive variant is issued as a separate query; inner product (equivalent to cosine similarity on unit vectors) retrieves the top-k candidates. Retrieval is correct if the anchor appears in the top-k. Metrics are MRR, Recall@k ($k = 1, 5, 10$), and NDCG@10, reported by query type and by script.

Four baselines are evaluated: **Levenshtein** [7, 10] ranks by character edit distance; **Double Metaphone** matches on phonetic codes with edit-distance tiebreaking; **BM25** [11] uses character trigrams; **Transliterate** converts the query to Latin via ICU `Any-Latin; Latin-ASCII` then applies Levenshtein ranking.

5 Results

5.1 Overall Performance

Table 1 presents the overall retrieval performance across all methods. The byte-level encoder significantly outperforms all baselines, achieving an MRR of 0.775 compared to 0.565 for the strongest baseline (Transliterate), a relative improvement of 37%. Edit-distance, phonetic-hashing, and BM25 baselines cluster near 0.09 MRR—their low overall scores reflect the dominance of script and combined queries in the evaluation set, on which these methods score near zero.

Table 1: Overall retrieval performance (11,974 entities, 470,772 queries).

Retriever	MRR	R@1	R@5	R@10	NDCG@10
Levenshtein	0.094	0.089	0.100	0.105	0.097
Double Metaphone	0.096	0.092	0.101	0.106	0.098
BM25	0.083	0.077	0.091	0.097	0.086
Transliterate	0.565	0.511	0.635	0.681	0.592
Byte encoder (ours)	**0.775**	**0.711**	**0.859**	**0.897**	**0.804**

5.2 Performance by Query Type

Table 2 breaks down MRR by query type. All methods perform comparably on phonetic queries, where query and document share the Latin script and character-level similarity is sufficient. Performance collapses for edit-distance and BM25 baselines on script and combined queries—they share no byte-level overlap with non-Latin documents, so retrieval is essentially random [1]. Transliterate recovers substantially on script queries (0.684) by converting all text to Latin before matching, but falls to 0.485 on combined queries where the fixed ICU mapping cannot account for phonetic variation in the query. The byte encoder maintains strong performance across all three types (0.937 / 0.827 / 0.738), demonstrating that the contrastive training signal successfully aligns phonetically equivalent byte sequences regardless of script.

Combined queries account for approximately 70% of the evaluation set and are also the most difficult setting (0.738 MRR vs. 0.937 for phonetic queries). As a result, they dominate the overall metric and explain why the aggregate MRR (0.775) appears significantly lower than performance on individual query types, highlighting the importance of reporting results by query type, as overall metrics alone can obscure task difficulty.

Table 2: MRR by query type.

Retriever	Phonetic	Script	Combined
Levenshtein	0.894	0.014	0.004
Double Metaphone	0.915	0.014	0.004
BM25	0.791	0.014	0.003
Transliterate	0.894	0.684	0.485
Byte encoder (ours)	**0.937**	**0.827**	**0.738**

5.3 Performance by Script

Table 3 reports R@10 per writing system. The model achieves R@10 above 0.95 on Arabic, Russian, Hebrew, Hindi, and Greek—scripts with relatively unambiguous romanization conventions where the LLM-generated training pairs are internally consistent. Greek achieves high performance (0.967 R@10) despite near-zero Wikidata supervision, owing to the near-bijective nature of Greek-to-Latin romanization: most Greek characters map consistently to a single Latin form. Arabic and Hebrew similarly benefit from consistent consonantal romanization, with vowel variation introducing only minor ambiguity that the model learns to tolerate. Japanese (0.870 R@10), despite higher Wikidata coverage than Arabic or Hebrew, underperforms due to multiple competing romanization systems (Hepburn, Kunrei-shiki, Nihon-shiki) and complex kanji readings that map the same character to entirely different sounds depending on context, producing inconsistent positive pairs during training.

Chinese (0.666) and Korean (0.728) are the hardest scripts: both suffer from severe romanization ambiguity (e.g., *Zhang / Chang / Cheung* for the same Chinese character; *Park / Pak / Bak* in Korean), causing the model to encounter conflicting phonetic mappings for the same entity during training [1]. Transliterate also struggles here (0.385 and 0.645), since ICU's rule-based system produces only one canonical romanization that may not match the query variant. Interestingly, BM25 performs slightly better on Chinese and Korean than other baselines—not due to phonetic matching, but because identical CJK characters may appear in both query and corpus when the query is already in the target script, producing incidental character n-gram overlap. This effect does not generalize to true cross-script retrieval and is absent for Latin queries against CJK corpora.

Table 3: R@10 by script.

Script	Lev.	D.Meta.	BM25	Translit.	Ours
Latin	0.944	0.952	0.891	0.944	**0.983**
Arabic	0.016	0.016	0.007	0.659	**0.967**
Russian	0.023	0.023	0.007	0.901	**0.974**
Chinese	0.007	0.007	0.018	0.385	**0.666**
Hebrew	0.019	0.019	0.016	0.433	**0.954**
Hindi	0.012	0.012	0.002	0.750	**0.973**
Greek	0.020	0.021	0.003	0.828	**0.967**
Korean	0.003	0.003	0.017	0.645	**0.728**
Japanese	0.005	0.005	0.014	0.598	**0.870**

5.4 Script Gap

We define the script gap as the R@10 difference between Latin and average non-Latin queries. All non-transliteration baselines exhibit gaps of 0.88–0.94 (Table 4): they retrieve well within Latin script but fail entirely across script boundaries, consistent with

prior findings on neural IR [1]. Transliterate narrows the gap to 0.275 by projecting all scripts to Latin. The byte encoder reduces it further to **0.096**—a 10× reduction over the Levenshtein baseline—while simultaneously improving Latin R@10 from 0.944 to 0.983.

The reduction in script gap is therefore driven not only by improved cross-script matching, but also by strong and consistent performance on scripts with stable romanization conventions, while the remaining gap (0.096) reflects irreducible ambiguity in CJK transliteration rather than a failure of the byte-level representation. Notably, the model also improves Latin R@10 from 0.944 to 0.983 over the Levenshtein baseline, confirming that the contrastive objective generalises within-script as well as across scripts.

Table 4: Script gap: R@10 difference between Latin and average non-Latin queries.

Retriever	Latin	Non-Latin (avg)	Gap
Levenshtein	0.944	0.013	0.931
Double Metaphone	0.952	0.013	0.939
BM25	0.891	0.010	0.881
Transliterate	0.944	0.669	0.275
Byte encoder (ours)	**0.983**	**0.887**	**0.096**

5.5 Indexing Efficiency

Table 5 reports the FAISS index ablation. HNSW [9] matches the recall of exact FlatIP search (R@10: 0.896 vs. 0.897) at 5.7× lower mean latency (0.03 ms vs. 0.17 ms), making it the preferred index for deployment. IVF-PQ reduces the index size by 96% (11.7 MB to 0.5 MB) at the cost of 6.4% R@10, a favourable tradeoff when scaling to millions of entities [5]. All indices build in under one second on 11,974 entities; at this scale exact search is already fast because the full index fits in CPU cache. At production scale (tens of millions of entities), HNSW remains the recommended choice, as its recall advantage over IVF-Flat (0.896 vs. 0.878) becomes more pronounced as the number of index partitions grows.

Table 5: FAISS index ablation (same model checkpoint).

Index	R@1	R@10	Latency (ms)	Size
FlatIP (exact)	0.711	0.897	0.17	11.7 MB
IVF-Flat	0.703	0.878	0.03	11.9 MB
HNSW	**0.711**	**0.896**	**0.03**	14.8 MB
IVF-PQ	0.604	0.833	0.06	0.5 MB

5.6 Limitations

The primary limitation is romanization ambiguity for CJK scripts: Chinese and Korean names have multiple valid romanizations across dialects and conventions, producing inconsistent training signal that the model cannot fully resolve. A second limitation is dataset composition—99.5% of positive pairs are LLM-generated, so evaluation quality is coupled to the quality of LLM transliteration; systematic transliteration errors would silently degrade both training and evaluation. Third, the pipeline generates non-Latin variants solely by transliterating Latin forms, never by augmenting within the target script itself. Native-script spelling variation—such as alternative Arabic orthographies or Korean spacing conventions—is therefore absent from training, potentially underestimating difficulty for native-script queries. Finally, evaluation is closed-world (corpus = 11,974 test entities only); real-world retrieval over millions of entities would yield lower absolute numbers due to increased distractor density.

6 Concluding Remarks

We presented a byte-level contrastive encoder for cross-script phonetic name retrieval, trained from scratch without any pretrained multilingual backbone. Operating directly on raw UTF-8 bytes, the model requires no tokeniser, no script detection, and no language-specific preprocessing, yet achieves 0.775 MRR and 0.897 R@10 on a benchmark spanning 8 non-Latin scripts—outperforming the strongest baseline (Transliterate) by 37% and reducing the R@10 script gap from 0.93 to 0.096, a 10× improvement.

The results confirm that byte-level tokenization provides a sufficient inductive bias for cross-script phonetic alignment: scripts with consistent romanization conventions (Arabic, Russian, Hebrew, Hindi, Greek) approach near-perfect retrieval, while CJK scripts remain the primary open challenge due to romanization ambiguity that produces conflicting training signal. HNSW indexing delivers exact-search recall at 5.7× lower latency, making the system viable for latency-sensitive deployment.

Future work should address CJK ambiguity directly—either through dialect-aware transliteration during data generation, or by incorporating script-specific supervision signals. Evaluating on open-world corpora with millions of distractors would further establish real-world robustness.

Acknowledgments

Course project for DSAIT4050 Information Retrieval, Q3 2025, Delft University of Technology. The authors acknowledge the use of computational resources of the DelftBlue supercomputer [3], provided by Delft High Performance Computing Centre.

References

[1] Aman Chari, Iadh Ounis, and Sean MacAvaney. 2025. Lost in Transliteration: Bridging the Script Gap in Neural IR. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. ACM. doi:10.1145/3726302.3730226

[2] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv preprint arXiv:2402.03216* (2024).

[3] Delft High Performance Computing Centre (DHPC). 2024. DelftBlue Supercomputer (Phase 2). https://www.tudelft.nl/dhpc/ark:/44463/DelftBluePhase2.

[4] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple Contrastive Learning of Sentence Embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*. Association for Computational Linguistics, 6894–6910. doi:10.18653/v1/2021.emnlp-main.552

[5] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572

[6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*. 6769–6781. doi:10.18653/v1/2020.emnlp-main.550

[7] Vladimir I. Levenshtein. 1966. Binary Codes Capable of Correcting Deletions, Insertions, and Reversals. *Soviet Physics Doklady* 10, 8 (1966), 707–710.
[8] Yihong Liu, Chunlan Ma, Haotian Ye, and Hinrich Schuetze. 2024. TransliCo: A Contrastive Learning Framework to Address the Script Barrier in Multilingual Pretrained Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. Association for Computational Linguistics, 2476–2499. doi:10.18653/v1/2024.acl-long.136
[9] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
[10] Gonzalo Navarro. 2001. A Guided Tour to Approximate String Matching. *Comput. Surveys* 33, 1 (2001), 31–88. doi:10.1145/375360.375365
[11] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.
[12] Joshua Robinson, Ching-Yao Chuang, Suvrit Sra, and Stefanie Jegelka. 2021. Contrastive Learning with Hard Negative Samples. In *International Conference on Learning Representations (ICLR 2021)*.
[13] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation Learning with Contrastive Predictive Coding. *arXiv preprint arXiv:1807.03748* (2018).
[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS 2017, Vol. 30)*.
[15] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models. *Transactions of the Association for Computational Linguistics* 10 (2022), 291–306. doi:10.1162/tacl_a_00461

Link to Repository

The code, dataset pipeline scripts, trained model checkpoint, and evaluation scripts are available at: https://github.com/vedant-jumle/cross-language-phonetic-text-alignment

Self-Assessment of Contributions

Vedant Vivek Jumle (5296196): Worked on leading and managing the project. Worked on modeling code. Worked on running all experiments on DelftBlue. Wrote the Abstract, Intro, Related work and conclusion section.
Carolyn Alcaraz (5680581): Contributed to the implementation of the data loading pipeline for large-scale multilingual name pairs. Assisted in developing the retrieval model and contributed to the analysis and presentation of the experimental results in the report.
Gonenc Turanli (5794765): Worked on implementing the step 2 of the LLM pipeline, worked on the evaluation script, worked on section 3 - Dataset of the report.
Ceylin Ece (5716950): Worked on implementing sampling and configuration components of the pipeline. Also, worked on developing baseline retrieval methods used in evaluation; these include Levenshtein, Double metaphone, transliteration, and BM25. Additionally, worked on Section 4 - Methodology of the report.

Declaration of Generative AI Usage

This project used generative AI tools as writing aids and for code assistance. Specifically, **Claude Sonnet 4.6** (Anthropic) was used to assist with writing refinement—improving clarity, grammar, and structure of sections drafted by the authors. The original intellectual contributions, research design, experimental methodology, and interpretation of results are entirely our own. All AI-assisted text was reviewed, revised, and approved by the authors before inclusion.

In accordance with the CEUR-WS GenAI Usage Taxonomy, our usage falls under *Writing Aid*—the tool refined prose written by the authors, but did not generate novel scientific content or conclusions.